DBMS / PRESENTATION

TRANSACTION ISOLATION LEVELS

How databases balance consistency, concurrency, and performance.

James McGuigan & Nitesh Kumar Chahar

PART 1 — ANOMALIES / PART 2 — ISOLATION CHOICES

What is isolation—and why do we need it?

Isolation defines what one transaction can see while another transaction is running.

THE IDEA

A set of database rules that controls whether one transaction can see another transaction’s changes.

WHY IT MATTERS

Many users' access data at once
Unfinished changes can cause confusing
Results Isolation reduces anomalies
without stopping all parallel work

GOAL: reliable results without stopping all concurrent work.

Isolation is a visibility contract

What Transaction B can see depends on the isolation level.

TRANSACTION A	TRANSACTION B
1 Read data	1 Read data
2 Update data	2 Update data
3 Commit changes	3 Commit changes

Isolation controls: uncommitted data • repeat reads • rows matching a range query

Four concurrency anomalies

Anomalies are surprising results caused by concurrent transactions.

01

DIRTY READ

Reading another transaction’s uncommitted change.

02

NON-REPEATABLE READ

Reading the same row twice and getting different values.

03

PHANTOM READ

Repeating a range query and getting a different set of rows.

04

LOST UPDATE

One transaction overwrites another transaction’s change.

What two concurrent transactions can do

01

DIRTY READ

T1 changes a balance to $1,000 but has not committed.

T2 reads $1,000.

T1 rolls back, the value was never permanent.

02

NON-REPEATABLE READ

T1 reads $500.

T2 changes it to $700 and commits.

T1 reads again and sees $700.

03

PHANTOM READ

T1 counts 3 matching orders.

T2 inserts another matching order.

T1 repeats the query and may count 4.

04

LOST UPDATE

Two shoppers read stock = 1 and both calculate stock = 0.

One update can overwrite the other.

Anomalies shown in SQL (1 of 2)

These snippets illustrate the timing problem; exact behavior depends on isolation and engine.

DIRTY READ

```
-- T1: update, but do not commit
UPDATE accounts SET balance = 1000
WHERE id = 1;

-- T2 reads 1000 before T1 commits
```

NON-REPEATABLE READ

```
-- T1 reads 500
SELECT balance FROM accounts WHERE id = 1;

-- T2 updates to 700 and commits
-- T1 reads again → 700
```

Anomalies shown in SQL (2 of 2)

Concurrent transaction timing determines the outcome.

PHANTOM READ

```
-- T1 finds 3 matching orders
SELECT COUNT(*) FROM orders
WHERE total > 100;

-- T2 inserts total = 150 and commits
-- T1 may now find 4
```

LOST UPDATE

```
-- T1 and T2 both read stock = 1
-- both calculate next_stock = 0
UPDATE items SET stock = 0
WHERE id = 7;

-- Last write can overwrite the other
```

Why have multiple isolation levels?

Different workloads need different balances of speed, waiting, and protection.

DASHBOARD

Changing counts may be acceptable.

A fresher view can matter more than a perfectly stable result.

STANDARD OLTP

Committed data with reasonable concurrency.

Read Committed is a common choice.

CRITICAL ALLOCATION

An impossible result is unacceptable.

Serializable offers the strongest protection.

RULE OF THUMB: use the lowest level that safely protects the business rule.

The four isolation levels

Typical SQL-standard summary; MySQL InnoDB details can depend on read type and locks.

LEVEL	DIRTY READ	NON-REPEATABLE	PHANTOM READ	TRADE-OFF
Read Uncommitted	Possible	Possible	Possible	Highest concurrency; weakest consistency
Read Committed	Prevented	Possible	Possible	Fresh committed view per read
Repeatable Read	Prevented	Prevented for snapshot reads	Depends on read type	InnoDB default; range locks may matter
Serializable	Prevented	Prevented	Prevented	Strongest protection; more waiting

The SQL standard and a database engine’s implementation are not always identical.

What each level means in practice

RU / READ UNCOMMITTED	**WEAKEST**	May see uncommitted values. All classic read anomalies are possible.
RC / READ COMMITTED	**COMMON CHOICE**	Dirty reads are prevented; repeated reads and range results may change.
RR / REPEATABLE READ	**INNODB DEFAULT**	Snapshot reads keep a consistent view; locking reads can behave differently.
S / SERIALIZABLE	**STRONGEST**	Transactions behave as if run one at a time; expect more waiting or conflicts.

Set the isolation level in SQL

Set the level before starting the transaction.

READ UNCOMMITTED

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

Unfinished data may be visible.

READ COMMITTED

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Reads committed data; later reads may change.

REPEATABLE READ

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

Stable snapshot reads; common InnoDB default.

SERIALIZABLE

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Strongest protection; may wait or retry.

Example Transaction: **START TRANSACTION;** → **SELECT ...;** → **COMMIT;**

Engine details matter

Do not assume every database engine handles every read the same way.

INNODB NOTES

Uses MVCC and locking strategies

Repeatable Read is the default in InnoDB

Snapshot reads and locking reads may behave differently

PRACTICAL CHECKLIST

Match isolation to the business rule

Test concurrent transactions

Check the query type and locking behavior

Expect waiting or retries at stronger levels

SET TRANSACTION applies to the next transaction when issued before START TRANSACTION.

Choose the guarantee your business rule needs

THREE IDEAS TO REMEMBER

1. Isolation controls what each transaction can see, and when.

2. Dirty reads, non-repeatable reads, phantoms, and lost updates are concurrency risks.

3. Isolation levels balance correctness, throughput, and workload needs.

USE THE LOWEST LEVEL

That “safely protects" the business rule.

QUESTIONS?

Thank You!